

Bases de datos orientadas a objetos, pruebas y documentación

3. *BD orientadas a objetos, pruebas y documentación*

Este apartado tiene dos partes bien diferenciadas: primero un bloque teórico sobre una categoría de bases de datos que no vas a usar en las prácticas —lo ves de forma conceptual, sin instalar ni configurar nada—, y después las pruebas que dan por buenas todas las piezas construidas hasta ahora.

Bases de datos orientadas a objetos puras

Una base de datos orientada a objetos almacena directamente el estado de los objetos, su identidad y las relaciones que mantienen entre ellos, sin descomponerlos previamente en el modelo relacional de filas, columnas y claves foráneas.

Esto no significa que copie literalmente un objeto Java tal como existe en la memoria del programa. El gestor utiliza su propia representación interna, pero su modelo lógico se basa en objetos y referencias, no en tablas relacionales.

Los tres modelos, uno junto a otro

	Relacional (Tema 1)	Objeto-relacional JSONB (este tema)	Orientada a objetos
Modelo de almacenamiento	Tablas, filas, columnas y relaciones	Modelo relacional, con tipos avanzados como jsonb	Objetos, identidades y referencias
Acceso desde Java	JDBC directamente o mediante un ORM como Hibernate	SQL/JDBC o, como aquí, Hibernate	API específica del gestor o estándares orientados a objetos
Lenguaje de consulta	SQL	SQL y operadores/funciones JSONB	Depende del gestor: OQL, JPQL, JDOQL o APIs propias
Adopción	Muy extendida	Muy extendida en gestores como PostgreSQL	Mucho menos habitual

La desventaja real de este modelo

La última fila resume la desventaja real de este modelo: menos documentación, menos gente con experiencia a la que preguntar, y el riesgo de quedarte atado a un gestor que puede dejar de mantenerse — por eso casi nadie elige hoy una BD orientada a objetos pura para un proyecto nuevo.

Al no existir la separación entre objetos y tablas relacionales, no hace falta un ORM objeto-relacional como Hibernate. Sigue siendo necesaria una API o un controlador que permita al programa comunicarse con el gestor, pero ya no tiene que traducir entidades a filas y claves foráneas.

OQL y gestores de referencia

OQL (Object Query Language) es uno de los lenguajes asociados históricamente a este modelo, pero no es una opción universal. Cada gestor puede ofrecer lenguajes y APIs diferentes. Algunos utilizan OQL; otros, como ObjectDB, permiten consultar directamente mediante JPQL o JDOQL sin traducir después la consulta a SQL.

db4o, ObjectDB y Versant son ejemplos representativos —algunos principalmente históricos— de este tipo de bases de datos. No todos utilizan el mismo lenguaje de consulta ni ofrecen las mismas APIs. No vas a instalar ninguno: este bloque es puramente conceptual.

Pruebas y documentación: test de capa

De vuelta al código: cómo se prueba y documenta lo construido en este tema.

Ya conoces el test de controller (PSP, Tema 1): con MockMvc, mockeando el service, prueba solo la capa HTTP — y aplica igual cuando la entidad lleva una columna JSONB. Existe también el test de service, que aísla la lógica de negocio mockeando el repositorio — pero ese se construye más adelante, en la Actividad 4.1; no hace falta todavía.

Test de integración real (con Testcontainers)

Los dos tests anteriores tienen algo en común: ninguno toca una base de datos de verdad, todo lo que hay debajo está mockeado.

Testcontainers es la librería que resuelve ese hueco — levanta contenedores Docker reales (una base de datos, una cola de mensajes, lo que haga falta) solo para la duración del test, y los destruye al terminar, sin dejar nada instalado en la máquina ni depender de ningún servicio externo compartido. No viene incluida en ningún starter usado hasta ahora — es la primera vez que aparece en el curso.

Por qué se fija Testcontainers 1.20.4

spring-boot-starter-parent gestiona también versiones de numerosas dependencias externas, entre ellas Testcontainers. Sin embargo, en este curso se fija expresamente Testcontainers 1.20.4, porque los módulos y los imports utilizados en las actividades corresponden a la línea 1.x.

Testcontainers 2 cambió los nombres de sus módulos y trasladó varias clases a paquetes nuevos. Utilizar directamente la versión gestionada por Spring Boot no sería un cambio transparente: habría que adaptar también las coordenadas Maven y algunos imports.

Idea clave: qué es un BOM

Un BOM (Bill of Materials, "lista de materiales") es un POM especial que coordina las versiones de un conjunto de artefactos compatibles, pero no añade por sí mismo esas librerías al proyecto.

spring-boot-starter-parent ya incorpora su propia gestión de dependencias. Al importar el BOM de Testcontainers 1.20.4, este proyecto fija explícitamente la versión de todos sus módulos y permite mantener las coordenadas e imports utilizados en las actividades.

El BOM va junto al <parent>

```
<properties>
  <testcontainers.version>1.20.4</testcontainers.version>
</properties>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.testcontainers</groupId>
      <artifactId>testcontainers-bom</artifactId>
      <version>${testcontainers.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Con el BOM importado, sin <version> individual

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <scope>test</scope>
</dependency>
```

Esto deja el terreno preparado para el resto del curso: cuando más adelante se añadan otros contenedores (MongoDB, RabbitMQ...), ninguno necesitará tampoco su propia <version> — todos quedan cubiertos por este mismo BOM.

Por qué esto funciona dentro del Dev Container

Testcontainers necesita arrancar contenedores de verdad — y el proyecto entero corre dentro de un contenedor (app) desde la Actividad 1.1. Funciona porque, ya desde entonces, se ha montado el socket de Docker del host dentro de app y se ha añadido la feature `docker-outside-of-docker`: el contenedor puede pedirle contenedores nuevos al mismo Docker que usa el sistema operativo, sin necesitar un Docker propio dentro del Docker.

Esa misma idea —la JVM, dentro de app, pidiéndole contenedores al Docker del host— es justo lo que puede fallar de dos maneras distintas, cada una en un momento distinto del proceso.

Primer fallo: Permission denied en el socket

Un `java.io.IOException / Permission denied` sobre `/var/run/docker.sock`, antes de crear ningún contenedor. En Docker Desktop ese socket pertenece al grupo `root`, y `docker-outside-of-docker` no puede añadir al usuario del contenedor a `root` por seguridad — se queda sin permiso para usarlo.

Solución: forzar los permisos del socket en cada arranque. Añadir a `devcontainer.json` (no a `docker-compose.yml`):

```
"postStartCommand": "sudo chmod 666 /var/run/docker.sock || true"
```

El `|| true` evita que el arranque falle si el comando no puede ejecutarse. Tras añadirlo, reconstruir el contenedor.

Segundo fallo: Connection refused con Ryuk

Testcontainers levanta, además del PostgreSQL de test, un contenedor auxiliar llamado Ryuk que limpia los contenedores al terminar — y ambos arrancan como contenedores hermanos en la red del host, no en la de app. El síntoma es Connection refused hacia una IP tipo 172.17.0.1.

Solución: indicar a Testcontainers una dirección que app sí pueda alcanzar. Añadir a `.devcontainer/docker-compose.yml`, en el servicio app:

```
environment:  
  - TESTCONTAINERS_HOST_OVERRIDE=host.docker.internal
```

Y reconstruir el contenedor. `host.docker.internal` es el nombre que Docker Desktop resuelve correctamente desde cualquier contenedor hacia los puertos publicados.

Con las dependencias resueltas: la clase de test

```
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
@Testcontainers
class LibreriaApiTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");

    @Autowired
    private MockMvc mockMvc;

    // ...

}
```


Qué hace cada anotación

`@Testcontainers + @Container` levantan un PostgreSQL real dentro de un contenedor Docker, solo para la duración del test — no una base de datos en memoria, no un mock.

`@ServiceConnection` conecta automáticamente ese contenedor a la aplicación de test, sin configurar manualmente la URL de conexión.

`@ActiveProfiles("test")` activa un perfil propio, distinto del dev de siempre — así se puede usar `ddl-auto: create-drop` (cada ejecución parte de una base de datos limpia) sin tocar la configuración diaria.

La anotación sola no basta: el perfil test debe existir

`@ActiveProfiles("test")` le dice a Spring qué perfil activar, pero no crea nada por sí sola. Tres propiedades sin valor impiden arrancar del todo: la contraseña del admin, el secreto JWT y la expiración del token — ninguna lleva valor por defecto. `spring.datasource` no hace falta escribirlo: eso ya lo resuelve `@ServiceConnection`. Crear `src/test/resources/application-test.yml`:

```
spring:
  config:
    activate:
      on-profile: test
  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: true

gamevault:
  admin:
    password: admintest123
  jwt:
    secret: un-secreto-de-test-de-al-menos-32-caracteres
    expiration-minutes: 60
```

Por qué esto da más confianza que mockear

Prueba el mapeo JSONB real contra un PostgreSQL real — una base de datos en memoria genérica (como H2) podría no soportar `jsonb_exists` exactamente igual, o ni siquiera soportar el tipo `jsonb` de PostgreSQL.

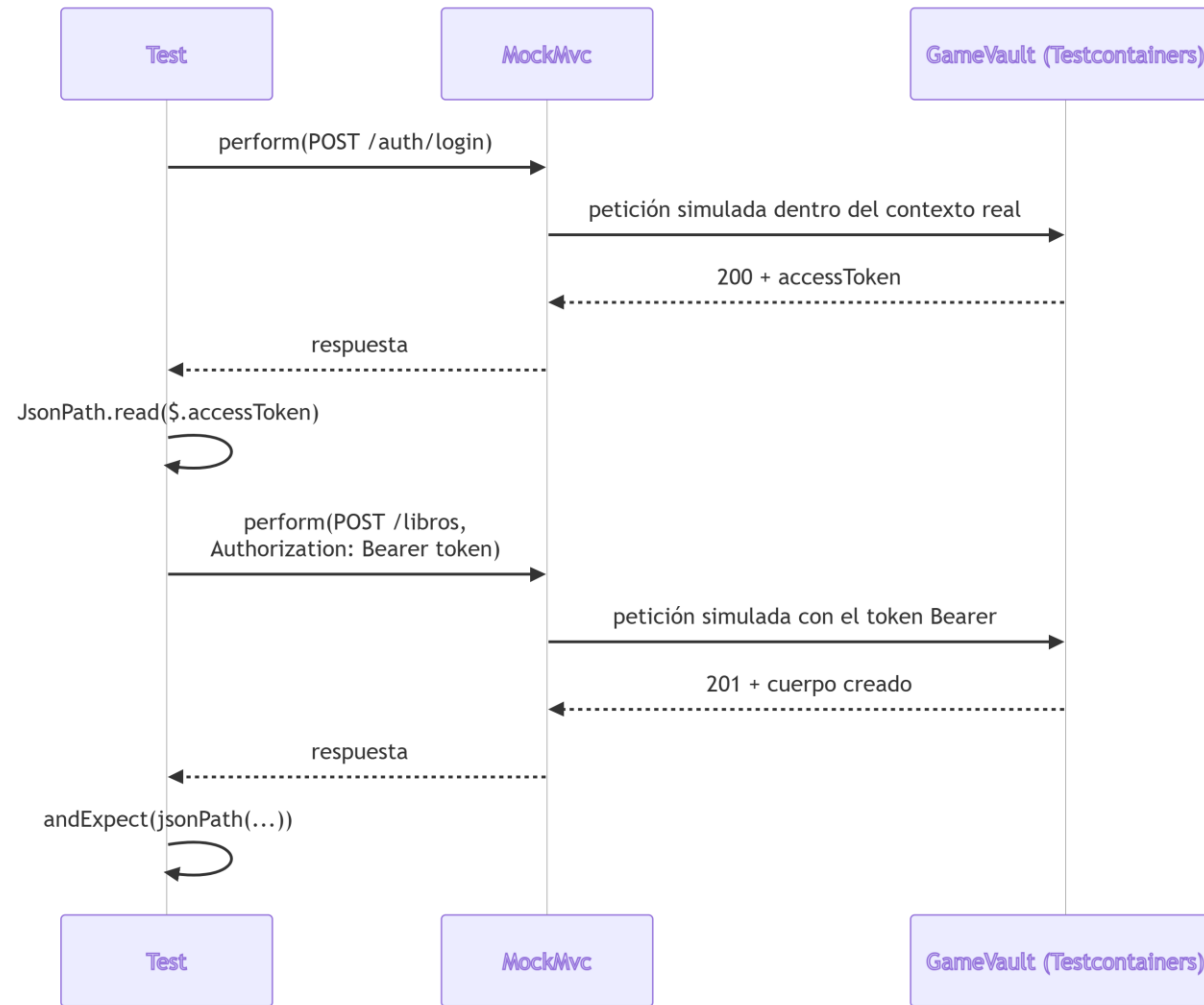
Un test con Testcontainers detectaría un error de mapeo que un test con mocks jamás vería, porque el mock nunca ejecuta SQL de verdad contra ningún motor.

Peticiones autenticadas dentro del test

Si la API tiene rutas protegidas —y a estas alturas del curso ya las tiene, con roles y todo—, el test debe recorrer el mismo flujo de autenticación y autorización de la aplicación: no hay ningún atajo que salte la seguridad "porque es un test".

MockMvc no abre una conexión HTTP real ni arranca un servidor. Construye peticiones y respuestas simuladas, pero las hace pasar por el DispatcherServlet, la cadena de filtros de seguridad, los controllers y el resto del contexto real de Spring. Por eso se puede hacer login, extraer el token y utilizarlo después en otra petición.

Login, token y petición autenticada



MockMvc hace pasar la petición por el contexto real de Spring — login, extracción del token con JsonPath, y una segunda petición autenticada con ese token.

El ayudante que hace login y devuelve el token

```
private String loginComoAdmin() throws Exception {
    String respuesta = mockMvc.perform(post("/api/v1/auth/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"username":"admin","password":"admintest123"}
            """))
        .andExpect(status().isOk())
        .andReturn().getResponse().getContentAsString();

    return JsonPath.read(respuesta, "$.accessToken");
}
```

`loginComoAdmin()` no lleva `@Test`: es un método ayudante que cualquier otro test de la clase puede utilizar. Requiere import `com.jayway.jsonpath.JsonPath`, ya incluido en `spring-boot-starter-test`.

El test que usa el token: crearLibro

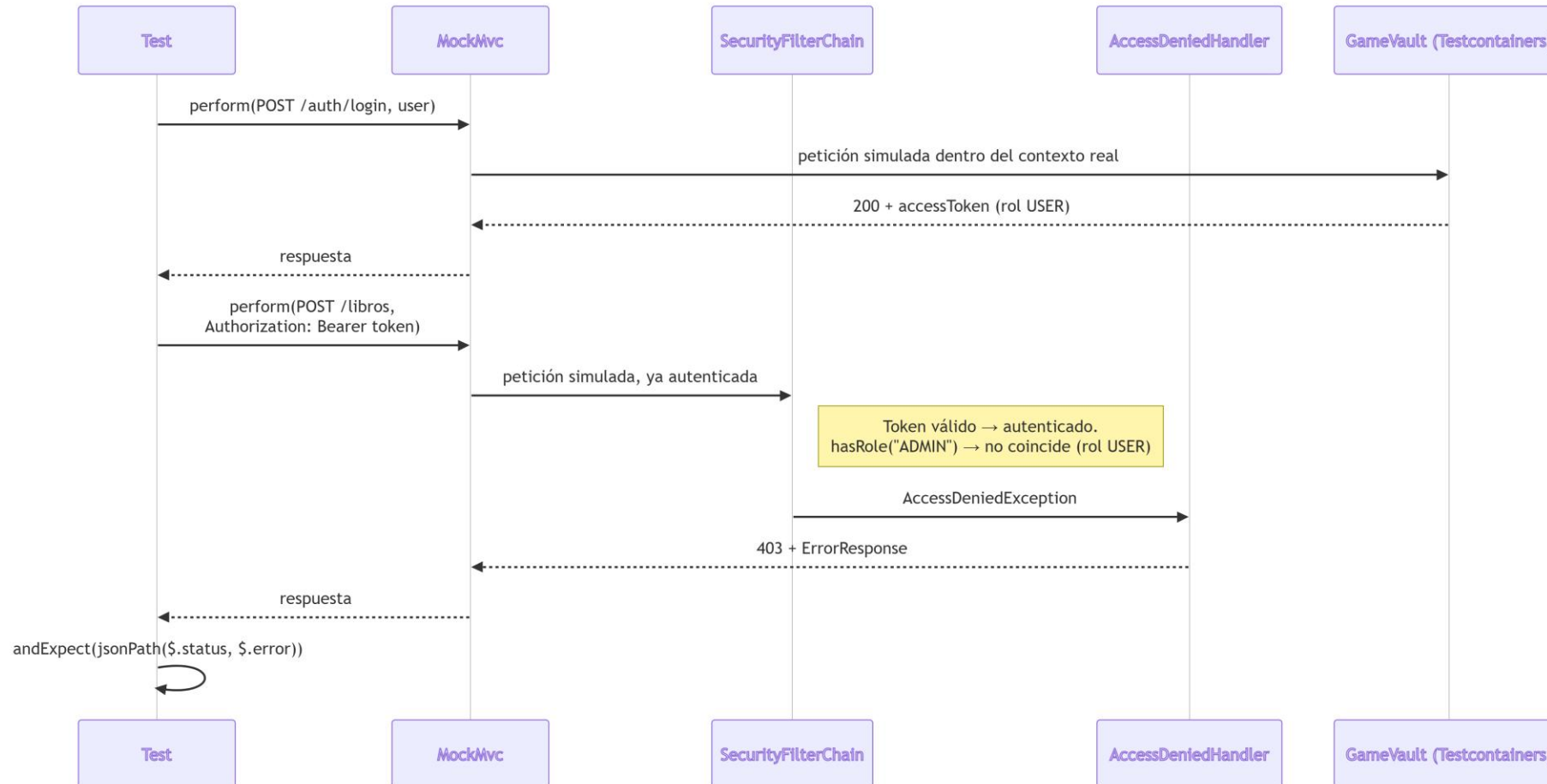
```
@Test
void crearLibro_DebeGuardarDetallesEdicion_CuandoEsValido() throws Exception {
    String adminToken = loginComoAdmin();

    mockMvc.perform(post("/api/v1/libros")
        .header("Authorization", "Bearer " + adminToken)
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"titulo":"Rayuela","detallesEdicion":
            {"editorial":"Sudamericana","paginas":634}}
            """))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.titulo").value("Rayuela"))
        .andExpect(jsonPath("$.detallesEdicion.editorial").value("Sudamericana"));
}
```

El test ejecuta el flujo completo de creación contra un PostgreSQL real y comprueba tanto el código de estado como el cuerpo, incluida la propiedad anidada detallesEdicion.editorial.

Comprobar también los casos de error

Un test de integración que solo prueba el camino feliz deja fuera la mitad del comportamiento real: qué pasa cuando la petición no debería tener éxito. Aquí la petición está autenticada —el token de user es válido—, pero no autorizada, así que ni siquiera llega al controller:



El test que reproduce el flujo 403

```
@Test
void crearLibro_DebeDevolver403_CuandoElUsuarioNoEsAdmin() throws Exception {
    String userToken = loginComoUsuario(); // igual que loginComoAdmin(), sin rol ADMIN

    mockMvc.perform(post("/api/v1/libros")
        .header("Authorization", "Bearer " + userToken)
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"titulo":"Rayuela","detallesEdicion":
            {"editorial":"Sudamericana","paginas":634}}
            """))
        .andExpect(status().isForbidden())
        .andExpect(jsonPath("$.status").value(403))
        .andExpect(jsonPath("$.error").value("No autorizado"));
}
```

No es casualidad que este test compruebe el cuerpo del error, no solo el código 403: dos errores distintos pueden compartir el mismo código de estado.

Por qué el cuerpo del error, no solo el status

Solo el cuerpo —con el `ErrorResponse` propio, no el genérico de Spring— dice cuál de los dos errores ha ocurrido de verdad. Quedarse solo con `status().isForbidden()` dejaría pasar una regresión real: por ejemplo, que el 403 siga llegando pero con el formato por defecto de Spring Security, porque alguien ha desconectado sin querer el `AccessDeniedHandler`.

Qué comprobar en un test de integración, casi siempre: el código de estado, la forma del cuerpo cuando lo hay (`jsonPath` sobre los campos que de verdad importan), y —cuando la petición debía cambiar algo— que ese cambio ha persistido de verdad, con una segunda petición que lo confirme.

Ideas clave

- Una BD orientada a objetos almacena estado, identidad y referencias sin descomponer en tablas relacionales; por eso no necesita un ORM objeto-relacional, aunque sí una API para comunicarse con el gestor.
- OQL es uno de los lenguajes asociados históricamente a este modelo, pero no es universal: cada gestor puede usar OQL, JPQL, JDOQL o APIs propias.
- Comparado con relacional puro y objeto-relacional (JSONB), lo orientado a objetos puro tiene poquísima adopción real — por eso este bloque es conceptual, no práctico.
- Un test de controller (PSP Tema 1) prueba la capa HTTP mockeando el service; un test de service mockea el repositorio (Actividad 4.1); un test de integración con Testcontainers levanta un motor real en Docker.

Ideas clave (cont.)

- Testcontainers detecta errores de mapeo real (como con jsonb) que un mock nunca podría detectar.
- Contra rutas protegidas, MockMvc reproduce el flujo real de login, filtros y autorización y extrae el token con JsonPath, aunque utiliza peticiones simuladas y no un servidor HTTP real.
- Los casos de error también se comprueban con jsonPath, campo a campo del ErrorResponse — no basta con el código de estado, porque dos errores distintos pueden compartirlo.
- Un test bien nombrado actúa como documentación ejecutable: si cambia el comportamiento que comprueba, el test falla. No sustituye toda la documentación escrita y solo documenta los casos que realmente cubre.